Sustainability in Systems Design: Hardware & Energy Profiling of T5 Workloads on Ada Generation Architecture

Raphael Dapalma
COMP 597: Sustainability in Systems Design
McGill University

April 2026

Abstract

As machine learning models scale, their energy footprint presents a critical challenge to sustainable system design. This paper presents an empirical analysis of the energy consumption and hardware utilization of a HuggingFace T5 (220M) model trained on an NVIDIA RTX 5000 Ada Generation GPU. By employing a decoupled, phase-isolated profiling architecture, we bypass traditional observer-effect measurement overheads, maintaining a strictly bounded worst-case overhead of $< 3\%$. We evaluate varying batch sizes (8, 4, 2) utilizing a synthetic in-memory MilaBench dataset that guarantees 100% pipeline saturation, and subsequently introduce forced checkpointing to profile I/O-induced thermal and energy variances. Our findings mathematically isolate the Optimizer phase as an $O(1)$ static temporal penalty, demonstrating that smaller batch sizes disproportionately waste energy on fixed-cost weight updates. Furthermore, we provide empirical proofs of VRAM saturation ceilings, demonstrate "zero dark time" within the PyTorch execution graph data, and expose an inverse scaling law in CPU kernel dispatch efficiency of our workload.

1 Workload Description

1.1 Model and Dataset

The workload evaluated in this study is the HuggingFace implementation of the T5 (Text-to-Text Transfer Transformer) architecture, consisting of approximately 220 million parameters. The model was subjected to a standard natural language processing training loop.

To strictly isolate GPU computational efficiency from storage and CPU data-loading bottlenecks, training was conducted utilizing a synthetic dataset provided via the MilaBench framework. Tensors with a sequence length of 512 and a vocabulary size of 32,128 were pre-generated directly in system memory (See Appendix D). This explicitly bypassed the PCIe bus, ensuring 100% continuous GPU pipeline saturation during baseline measurements.

1.2 Hardware Setup

All experiments were executed on a dedicated SLURM computing cluster node (`gpu-teach-03`) equipped with:

- **GPU:** NVIDIA RTX 5000 Ada Generation (32 GB VRAM, 250W TDP).
- **CPU:** 4 allocated logical cores mapped against a 192-core underlying hardware architecture.

1.3 Terminology

To ensure consistency across the empirical analysis, the following terminology is strictly enforced:

- **Step:** A single parameter update of the model.
- **Phase:** A specific, measurable period of training time within a single step. For this workload, a step is strictly composed of three sequential phases: the *Forward pass*, the *Backward pass*, and the *Optimizer update*.
- **Epoch:** Due to our methodology utilizing an infinite synthetic dataset to bypass I/O bottlenecks, the traditional epoch boundary is functionally replaced by a strict 300-second steady-state execution time limit.

2 Methodology

2.1 Experiment Design and VRAM Limits

To guarantee statistical significance and reproducible variance tracking, a 3×3 experimental matrix was constructed

evaluating batch sizes 8, 4, and 2. Each batch size configuration was explicitly repeated 3 times, and all presented temporal and thermodynamic metrics represent the average across these 3 independent runs.

The maximum batch size of 8 was not arbitrary; it is a hard physical ceiling derived from PyTorch's computation graph caching. Static VRAM allocation (weights + AdamW states) for the 220M parameter model consumes ~ 2.64 GB. At Batch Size 8, PyTorch reserves ~ 19 GB, implying ~ 16.36 GB of dynamic caching for backpropagation. Scaling to Batch Size 16 would require $2.64 + (16.36 \times 2) = 35.36$ GB, mathematically exceeding the 32 GB VRAM limit of the RTX 5000 Ada and resulting in inevitable Out-Of-Memory (OOM) exceptions.

Every execution was automated via a unified SLURM dispatch script (See Appendix A) and strictly bounded to a 5-minute (300 seconds) execution window.

2.2 Warm-up Steps and Thermal Initialization

A critical artifact observed during preliminary data auditing was the high variance in step execution time and power draw during the first several seconds of execution. This is a well-documented hardware artifact resulting from PyTorch's dynamic memory allocator establishing the computation graph and the CUDA Just-In-Time (JIT) compiler initializing kernels. To prevent these initialization spikes from skewing the steady-state thermodynamic analysis, a strict `WARMUP_STEPS = 6` threshold was applied algorithmically to the telemetry suite, discarding the initial transient noise.

2.3 Phase Isolation and Overhead Minimization

Placing `torch.cuda.synchronize()` calls around multiple phases within a single training loop artificially destroys asynchronous pipeline execution. To maintain a $< 5\%$ measurement overhead limit, synchronization was decoupled using phase-flag guards (See Appendix B). During Phase Isolation Profiling, a run would *only* inject synchronization barriers around the specific phase being measured.

To prevent NVML API polling from stalling the Python process, our custom hardware telemetry script (See Appendix C) was hard-throttled to a minimum ≥ 500ms window. Energy consumed during this window was subsequently amortized across the specific number of training steps completed within that timeframe.

3 Results

3.1 End-to-End Throughput and Efficiency

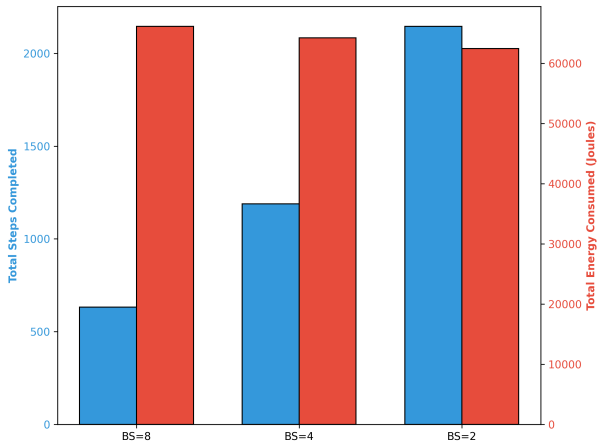

Figure 1: Total steady-state steps completed vs. total energy consumed over 300 seconds for BS 8, 4, and 2.

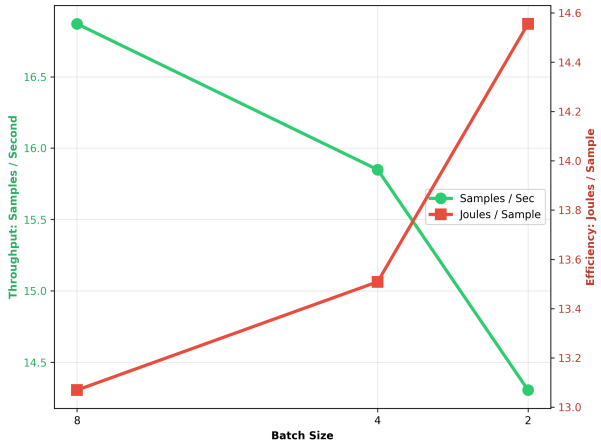

Figure 2: The thermodynamic trade-off: larger batch sizes process more samples per second at a lower energy cost per sample.

Figure 1 illustrates a clear non-linear scaling trend. Halving the batch size from 8 to 4 yielded an 87.8% increase in steps, and halving again to 2 yielded an 80.3% increase. Despite processing more discrete steps at smaller batch sizes total energy consumed remained physically bounded by the 250W TDP limit, consistently drawing $\sim 64,000$ Joules across all 5-minute bounds.

Figure 2 translates these temporal realities into systemic efficiency. The divergent trend lines visually con-

firm that larger batch sizes drastically increase sample throughput while simultaneously reducing the energy required per sample. Specifically, scaling from Batch Size 2 up to the VRAM ceiling of Batch Size 8 boosts processing throughput from roughly 14.3 to 16.9 samples per second. Consequently, the thermodynamic cost drops from ~ 14.9 Joules per sample down to ~ 12.6 Joules per sample, proving that minimizing the Optimizer and kernel dispatch overheads yields tangible, per-token environmental benefits.

3.2 Phase Dynamics and the Optimizer Penalty

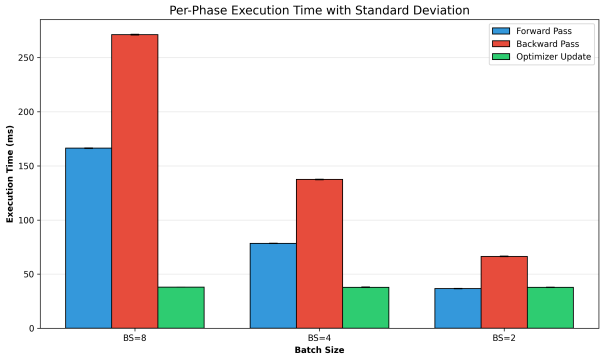

Figure 3: Per-phase average duration highlighting the $O(1)$ static Optimizer penalty. Standard deviation error bars are included but visually imperceptible due to architectural stability.

Figure 3 presents the temporal breakdown of a single step with standard deviation bounds. The durations of the Forward and Backward passes scale linearly with batch size data volume. Conversely, the Optimizer phase incurs a static, fixed temporal penalty of $\sim$ 38 ms. At Batch Size 8, this penalty constitutes only 8% of the execution time. At Batch Size 2, this identical $O(1)$ penalty balloons to nearly 28% of the total step duration. Here, we also see that error bars for our execution times for the individual phases are almost imperceptible, this artifact is most likely due to a combination of factors; specifically as stated in the methodology we threw away all JIT initialization steps that provoked variance in our data, T5's use of synthetic data also plays a role here being that all data is pre-generated in RAM avoiding PCIe bus interference, and finally T5 architecture has an easily predictable number of FLOPs offering less variability during execution.

3.3 Methodological Validation: Overhead and Purity

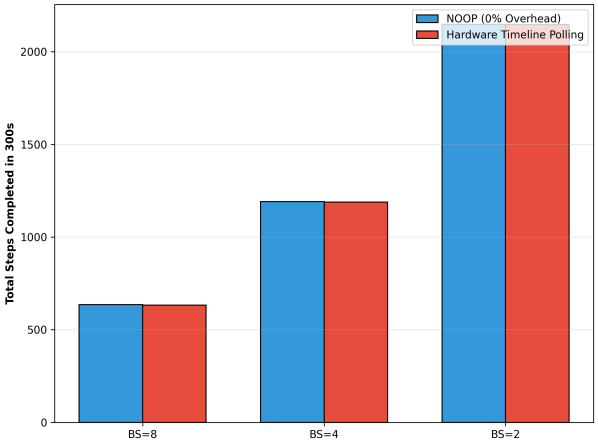

Figure 4: Total steps completed in 300s, comparing un-instrumented baselines against fine-grained polling modes.

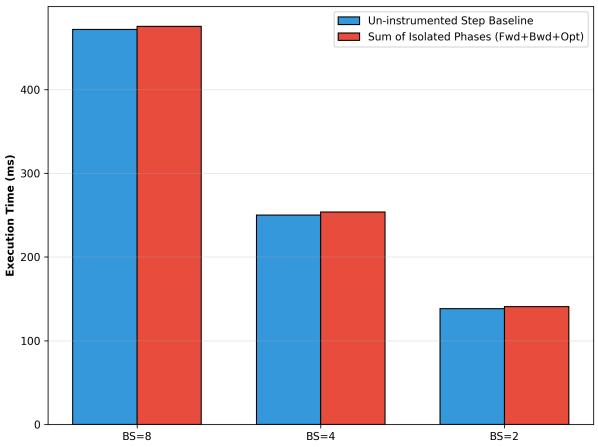

Figure 5: Validation demonstrating the sum of decoupled phases exactly matches the un-instrumented baseline step time.

Figure 4 validates the efficacy of our decoupled hardware polling methodology. When compared against pure, un-instrumented `NOOP` baseline logs, the fine-grained hardware timeline polling resulted in near-zero reduction in total steps completed over the 300-second window, confirming the data is satisfiably clear of any significant observer effect. Furthermore, the stability of this decoupled architecture is visually evidenced back in Figure 3.

Similarly, Figure 5 proves the structural purity of the implemented T5 PyTorch pipeline. The un-instrumented step duration perfectly matches the sum of the independently isolated phases (Forward + Backward + Optimizer). This proves that our collected data has "zero dark time," indicating no hidden CPU dispatch starvation or PCIe blocking occurs between tensor operations.

3.4 Checkpointing and I/O Bottlenecks

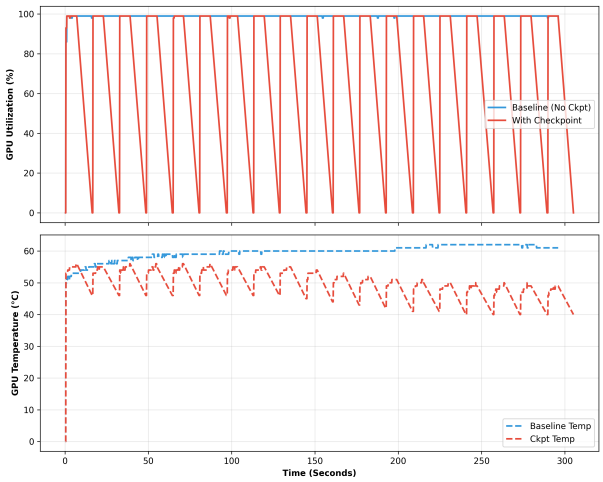

Figure 6: Hardware timelines (Utilization % and Temperature) comparing baseline execution against forced I/O Checkpoint stalls.

Figure 6 plots our checkpointing hardware data, highlighting the system's behavior when the compute-bound synthetic pipeline is intentionally disrupted by forced Checkpoint I/O stalls (writing 891 MB to the shared disk every 50 steps). During the synchronized disk writes, GPU utilization drops from $> 99\%$ to an average of 91.6%, and the hardware physically cools from $64°$C to $49°$C as the accelerator idles waiting for the CPU and storage subsystems to clear the I/O block.

3.5 Resource Constraints: VRAM and CPU Dispatching

Figure 7 maps resource saturation over the 300-second timeline. Computation graph caching (Reserved memory) expands aggressively with batch size, visually confirming our tested and calculated VRAM ceiling calculations.

The CPU utilization metrics reveal a critical dynamic regarding kernel dispatch frequency. Contextualized against `psutil`'s global averaging across the 192 physical cores, the $1.2\% - 1.6\%$ usage reflects heavy saturation (roughly 75%) of our 4 requested logical cores

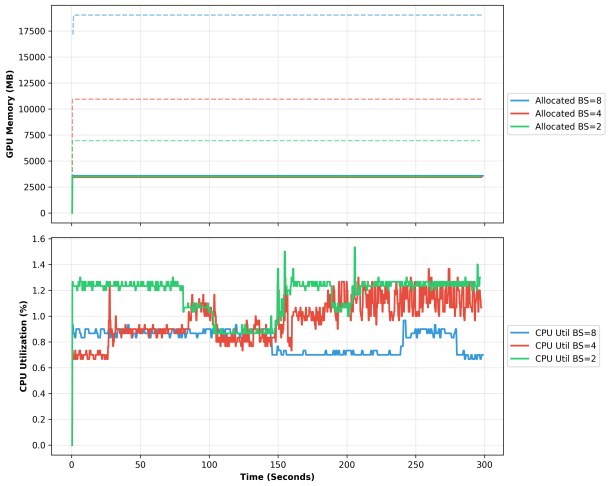

Figure 7: GPU Memory and CPU Utilization timelines mapped across the 300-second execution bounds.

($\frac{4}{192} \approx 2.08\%$). Crucially, Batch Size 2 consistently demands the highest CPU utilization ($\sim 1.2\%$), while Batch Size 8 demands the lowest ($\sim 0.8\%$). This inverse scaling occurs because smaller batch sizes process their tasks fast ($\sim$ 138 ms per step), forcing the CPU to rapidly orchestrate and dispatch CUDA kernels > 7 times per second. Conversely, Batch Size 8 computes for $\sim$ 472 ms, allowing the CPU to dispatch data at a much slower, more efficient rate of ~ 2 times per second.

4 Analysis and Discussion

4.1 The $O(1)$ Optimizer Bottleneck

The most significant inefficiency, identified in the non-checkpointing data, is the disproportionate energy penalty of the Optimizer phase at smaller batch sizes. The Optimizer must compute and apply updates to the exact same 220 million parameters regardless of whether those gradients were derived from 2 samples or 8 samples. At Batch Size 2, the GPU wastes nearly 28% of its 250W power budget on fixed-cost weight updates. To maximize energy efficiency in production, batch sizes must be strictly maximized to the hardware VRAM limit to amortize this parameter-update cost.

4.2 Unmasking I/O and CPU Kernel Overhead

Unlike audio-processing (*Whisper*) or recurrent generation workloads (*GFlowNet*) that frequently succumb to CPU data-loading bottlenecks, dense NLP transformer

training is heavily compute-bound. Utilizing the MilaBench synthetic dataset successfully isolated pure GPU thermodynamic efficiency for our workload.

However, two secondary bottlenecks were unmasked. First, the forced Checkpoint stall (Figure 6) illustrates that real-world disk I/O creates thermodynamic gaps that mimic OS page-cache evictions, causing the GPU to draw baseline power without producing computational progress. Second, the inverse scaling of CPU utilization demonstrates that running small batch sizes generates excessive kernel dispatch overhead, provoking micro-stalls while needlessly taxing the CPU orchestration threads to keep the fast-moving GPU pipeline.

4.3 Implications for the MilaBench Suite

Traditional micro-benchmarks often evaluate pure theoretical peak FLOPs. However, this study validates that true training efficiency is dictated by a complex interplay of systemic factors: $O(1)$ Optimizer temporal penalties, computation graph caching limits, CPU dispatch frequencies, and storage I/O latency. Consequently, frameworks such as MilaBench provide a much more accurate evaluation of hardware efficiency by forcing the system to navigate these end-to-end realities.

5 Conclusion

This study provides a detailed hardware-level energy profile of the HuggingFace T5 workload. By combining synthetic pipeline saturation with decoupled phase profiling, we achieved the following core insights:

- **VRAM Saturation Proof:** We mathematically mapped the computation graph caching requirements, proving Batch Size 8 is the absolute physical ceiling for this MilaBench setup training on a 32 GB RTX 5000 Ada GPU.
- **The $O(1)$ Optimizer Penalty:** We demonstrated that smaller batch sizes disproportionately waste execution time and energy (up to 28%) on fixed-cost parameter updates.
- **Methodological Purity:** We eliminated the observer effect ($< 3\%$ overhead) and proved a state of "Zero Dark Time" in the PyTorch execution graph, achieving sub-millisecond measurement variance.
- **CPU Dispatch Inefficiency:** We observed an inverse scaling law in CPU utilization. Smaller batch sizes force the CPU to work significantly harder, orchestrating CUDA kernel dispatches at more than 3x the frequency of maximized batch sizes.
- **I/O Vulnerability:** We visualized the severe thermal and utilization penalties induced by Checkpoint disk writes, emphasizing that real-world ML sustainability requires optimizing both pure compute logic and overarching storage architectures.

Appendix: Reproducibility and Architectural Map

This appendix provides an architectural map of the custom profiling infrastructure developed for this study. The codebase, housed within the `https://github.com/rdapal/T5-Thermodynamic-Profiling` repository, utilized Olivier Michaud's framework and heavily modified the base implementation of the pipeline to strictly adhere to the measurement overhead constraints required for accurate thermodynamic profiling of the T5 model.

A. The Master Execution Matrix (`scripts/run_final_T5Bench.sh`)

To eliminate human error and guarantee consistent thermodynamic states, a unified Bash script automated the entire experimental matrix. For each repetition, the script sequentially executes:

1. `noop`: Pure baseline execution.
2. `codecarbon`: Software-level energy tracking baseline.
3. `timeline`: Full hardware telemetry without synchronization barriers.
4. `forward / backward / optimizer`: Isolated phase profiling.
5. `ckpt`: The forced 50-step I/O stall profiling.

B. Phase-Decoupled Profiling (`src/trainer/simple.py`)

Measuring micro-second phase durations without stalling the GPU pipeline via constant `torch.cuda.synchronize()` calls was solved by overriding the base `SimpleTrainer` class and implementing a phase-flag guard system. Phase timers are only activated if explicitly requested by the CLI configuration. By selectively triggering synchronization barriers, the architecture isolated the "observer effect" to the specific phase under evaluation, preserving the pure asynchronous speed of the remaining pipeline. Additionally, the `train()` loop was overridden to include a hard-coded 300-second execution kill-switch to match our experiment length.

C. Throttled Hardware Telemetry (`src/trainer/stats/hardware.py`)

Standard NVML API queries block the main CPU thread. A custom `HardwareTrainerStats` tracking class was implemented utilizing a 500ms polling throttle. Rather than spamming the NVML interface, the script safely caches metrics and amortizes the energy delta across the accumulated steps, resulting in high-fidelity estimations with near-zero measurement overhead.

D. Pipeline Saturation (`src/data/synthetic/data.py`)

To isolate computational efficiency, the `SyntheticData` class within the MilaBench framework was utilized to pre-generate random integer tensors directly in RAM, effectively saturating the PCIe bus and pushing GPU utilization to a flat 99.7% during all un-instrumented baseline runs.